
**TED UNIVERSITY**

**CMPE 492 / SENG 492 Senior Project II**

# Center of Gravity Detection, Information, and Balancing System in Aircraft

**Final Report**

**15.05.2026**

**Team Members:**

Selin Göç, Computer Engineering

Elif Sude Memiş, Software Engineering

Sude İpekci, Computer Engineering

Yasin Anber, Computer Engineering

**Supervisor:**

Prof. Dr. Hakkı Gökhan İLK

**Jury Members:**

Dr. Ali BERKOL

Dr. Ayşe Yasemin SEYDİM

# Table of Contents

**ABSTRACT** ..... 3

**1. INTRODUCTION** ..... 3

**2. SYSTEM OVERVIEW** ..... 3

**3. INTERFACE DOCUMENTATION GUIDELINES** ..... 3

**3.1 High-Level Architecture** ..... 3

**3.2 Low-Level Design** ..... 4

**3.3 CG Calculation Algorithm** ..... 4

***3.3.1 Load Cell Positions*** ..... 4

***3.3.2 CG Formula*** ..... 4

***3.3.3 Averaging for Noise Reduction*** ..... 5

***3.3.4 Update Rate*** ..... 5

**3.4 Control Mechanism** ..... 5

***3.4.1 Deviation Detection*** ..... 5

***3.4.2 Required Counterweight Displacement*** ..... 5

***3.4.3 Motor Step Calculation*** ..... 6

***3.4.4 Boundary Check*** ..... 6

***3.4.5 Closed-Loop Termination*** ..... 6

**4. IMPLEMENTATION** ..... 7

**4.1 Hardware Implementation** ..... 7

**4.2 Software Implementation** ..... 7

**5. TOOLS AND TECHNOLOGIES USED** ..... 7

**6. LITERATURE REVIEW AND RESOURCES** ..... 7

**7. TEST RESULTS** ..... 8

**7.1 Test Plan Summary** ..... 8

**7.2 Test Results** ..... 9

**8. IMPACT OF ENGINEERING SOLUTIONS** ..... 19

**9. CONTEMPORARY ISSUES** ..... 19

**10. FUTURE IMPROVEMENTS** ..... 19

**11. CONCLUSION** ..... 20

**12. GITHUB REPOSITORY** ..... 22

**14. REFERENCES** ..... 22

## ABSTRACT

This project presents the design and implementation of an Aircraft Center of Gravity (CG) Detection and Balancing System aimed at improving flight safety and operational efficiency. The system utilizes load cell sensors to measure weight distribution across multiple points, with data acquisition handled through the HX711 amplifier and processed by an Arduino-based microcontroller. A CG calculation algorithm determines the aircraft's center of gravity in real time. Based on this calculation, a step motor-driven mechanism with a lead screw system actively adjusts a movable mass to restore balance along both X and Y axes. The system also includes a user interface for monitoring CG values and system status. Experimental results demonstrate that the system can accurately detect CG shifts and perform corrective balancing actions. This project contributes to the development of intelligent, real-time monitoring and control systems in aviation, with potential applications in safety-critical environments.

## 1. INTRODUCTION

Maintaining the correct center of gravity (CG) is critical for the safe and efficient operation of aircraft. An improper CG distribution can lead to instability, increased fuel consumption, and in extreme cases, loss of control. Traditionally, CG calculations are performed manually or through static systems, which may not account for real-time variations in weight distribution.

The motivation behind this project is to develop an automated system capable of continuously monitoring and correcting CG imbalances. By integrating sensors, embedded systems, and mechanical actuators, the proposed solution provides a real-time and dynamic approach to aircraft balancing.

The main objective of this project is to design and implement a system that can accurately detect the center of gravity using sensor data and actively correct imbalances through a motorized mechanism. The scope of the project includes sensor integration, data processing, CG calculation, control system design, and implementation of a balancing mechanism.

## 2. SYSTEM OVERVIEW

The Aircraft CG Detection and Balancing System consists of sensor, processing, control, and actuation subsystems working together to achieve real-time balancing.

Load cell sensors are placed at critical support points to measure the weight distribution. These analog signals are amplified and digitized using the HX711 module. The processed data is then sent to an Arduino microcontroller, which calculates the center of gravity using predefined equations.

Based on the computed CG position, the system determines whether an imbalance exists. If necessary, step motors drive a lead screw mechanism to reposition a counterweight along X and Y axes to restore balance. A user interface displays real-time CG data and system status.

## 3. INTERFACE DOCUMENTATION GUIDELINES

### 3.1 High-Level Architecture

The system is composed of four main subsystems: sensing, data acquisition, processing, and actuation. The sensing subsystem collects weight data using load cells. The acquisition subsystem (HX711) converts

analog signals into digital form. The processing unit (Arduino) computes CG and determines corrective actions. The actuation subsystem physically adjusts the system using motors and a lead screw mechanism.

## 3.2 Low-Level Design

The load cells operate based on strain gauge principles, converting mechanical force into electrical signals. These signals are amplified by the HX711, a 24-bit ADC designed for high-precision weight measurements.

The Arduino Uno (ATmega328P) processes incoming data, executes the CG calculation algorithm, and generates control signals for the actuators. Step motors are used due to their precise position control capabilities, and they are coupled with a lead screw system to enable linear motion in both axes.

The user interface is implemented via serial communication, allowing real-time monitoring of CG values and system feedback.

## 3.3 CG Calculation Algorithm

### *3.3.1 Load Cell Positions*

Three full-bridge strain gauge load cells are used, placed beneath the aircraft platform, with one load cell per point where each landing gear touches the ground. The position of each load cell relative to the reference datum is known and programmed into the Arduino software using constant values:

| Load Cell | Location | X Coordinate (mm) | Y Coordinate (mm) |
|---|---|---|---|
| LC1 | Front | 0.0 | 14.5 |
| LC2 | Left back | Calculation not required | 10.5 |
| LC3 | Right back | Calculation not required | 10.5 |

### *3.3.2 CG Formula*

The calculation of center of gravity is done via weighted moment approach in which the force values measured by load cells are multiplied with their positions respectively. The formulas used to calculate these values are shown in Eq. (1) and Eq. (2).

$$CG_x = \frac{F_1X_1 + F_2X_2 + F_3X_3}{F_1 + F_2 + F_3} \quad (1)$$

$$CG_y = \frac{F_1Y_1 + F_2Y_2 + F_3Y_3}{F_1 + F_2 + F_3} \quad (2)$$

Where $F_1$, $F_2$, and $F_3$ are the digital readings of force sensed by load cells LC1, LC2, and LC3 respectively, and $(X_1,Y_1)$, $(X_2,Y_2)$, $(X_3,Y_3)$ are their respective coordinates.

#### *3.3.3 Averaging for Noise Reduction*

Since there would be some noise in the raw load cell values, the program takes the average of N readings from each load cell for each cycle using the arithmetic average and then applies the CG equation. This helps reduce the impact of noise spikes in the computation of the CG values.

#### *3.3.4 Update Rate*

The program runs in the main loop on the Arduino board. One complete iteration, which involves three sensor reads and calculation of CG coordinates, is completed within the specified 50 ms period, as required in NFR1. These calculated values for $CG_x$ and $CG_y$ are then provided to the controller algorithm and simultaneously sent through serial communication.

### 3.4 Control Mechanism

#### *3.4.1 Deviation Detection*

After every iteration of the CG computation, the program calculates the difference between the current value of the CG and the optimum stored value. The corresponding calculations are presented in Eq. (3) and Eq. (4) below:

$$\Delta X = CG_{x,\text{current}} - CG_{x,\text{opt}} \quad (3)$$

$$\Delta Y = CG_{y,\text{current}} - CG_{y,\text{opt}} \quad (4)$$

The balancing procedure only initiates itself if one of the two errors calculated above goes beyond the limits set by NFR2 (±2% of the operational area). Otherwise, if both values fall within the limits, then the status of the system is set to "STABLE".

#### *3.4.2 Required Counterweight Displacement*

The amount of counterweight displacement needed for balancing the detected error is based on the concept of moment equilibrium. Displacement of the counterweight having a mass of $W_{cw}$ will create a reverse moment to oppose the error as shown in Eq. (5) and Eq. (6):

$$d_x = \frac{W_{total} \cdot \Delta X}{W_{cw}} \quad (5)$$

$$d_y = \frac{W_{total} \cdot \Delta Y}{W_{cw}} \quad (6)$$

Where $W_{total}$ is the total weight indicated by the three load cells and $W_{cw}$ is the constant mass of the counterweight, both expressed in grams.

### *3.4.3 Motor Step Calculation*

Eq. (7) and Eq. (8) show the step motor rotations required for displacement calculations, computed by the Arduino according to the lead screw pitch (3 mm per rotation):

$$\text{Rotations}_x = \frac{d_x}{3} \quad (7)$$

$$\text{Rotations}_y = \frac{d_y}{3} \quad (8)$$

The appropriate PWM values are then used to control the two step motors. The motion occurs in small incremental steps to avoid mechanical overshooting and oscillation, while the load cell values are sampled at each iteration until an equilibrium is achieved.

### *3.4.4 Boundary Check*

Prior to executing the move, the system checks whether the required displacement exceeds the mechanical displacement limits (Max_Disp_X and Max_Disp_Y from the BalancingMechanism class). Should the required displacement exceed these values, the system will remain stationary at its maximum possible displacement and will send an alert message to the user interface.

### *3.4.5 Closed-Loop Termination*

The sensing, calculating, actuating, and sensing again process continues in a continuous loop until both |ΔX| and |ΔY| values become lower than the set threshold. This will result in a "STABLE" status output through the serial communication port and an indication on the interface that the balance has been achieved.

# 4. IMPLEMENTATION

## 4.1 Hardware Implementation

The hardware setup includes multiple load cells mounted under the structure to capture weight distribution. The HX711 module is connected to the load cells for signal amplification and digitization. The Arduino Uno acts as the central controller.

Step motors are connected to a lead screw mechanism that allows controlled linear movement. The mechanical structure is designed to simulate aircraft balancing behavior.

## 4.2 Software Implementation

The software is developed using the Arduino IDE in C/C++. The program consists of several modules including sensor data acquisition, CG computation, decision-making logic, and motor control.

The system continuously reads sensor values, processes the data, calculates CG, and triggers the control mechanism if needed. Serial communication is used for debugging and monitoring purposes.

# 5. TOOLS AND TECHNOLOGIES USED

The project utilizes a combination of hardware and software tools:

- Arduino IDE for embedded programming
- C/C++ for microcontroller development
- HX711 library for sensor interfacing
- GitHub for version control and collaboration
- Serial communication tools for monitoring

These tools enabled efficient development, testing, and integration of the system.

# 6. LITERATURE REVIEW AND RESOURCES

The development of the proposed system draws its roots from various technical literature sources.

According to FAA Weight and Balance Handbook (FAA-H-8083-1B, 2016) [2], the standard moment-arm method is used to calculate the center of gravity, which represents the mathematical foundation of the algorithm in this project. The manual also indicates that even minor variations in the CG position by several percent of the permissible range can change the longitudinal stability, thus explaining the accuracy criteria set in NFR2.

The book Raymer's Aircraft Design: A Conceptual Approach [cited in the Analysis Report] also reinforces that the location of the CG is among the most crucial factors in the initial phase of aircraft design, and that the process of manually moving the CG through trial and error while performing balancing is prone to mistakes and consumes a lot of time.

According to Abdelhafiz and El-Sayed (2020), automated CG measuring systems that use load cells for small aircraft are able to perform measurements with an accuracy rate of around ±1–2%.

The HX711 data sheet helped to determine the signal acquisition circuitry: the module's 24-bit resolution along with the PGA gain of 128 (channel A) is ideal for amplification of the relatively low-millivolt signals from full bridge load cell strain gauges. This solution was implemented in the prototype directly.

The use of lead screw actuators was confirmed in Soneji et al. (2013) [5], which demonstrated that lead screws of 3 mm pitch were enough for millimetric resolution of counterweight movement under the control of a step motor.

# 7. TEST RESULTS

## 7.1 Test Plan Summary

Testing was conducted across seven phases as defined in the Test Plan ReporT: Hardware Unit Testing, Software Unit Testing, Integration Testing, System Testing, Performance Testing, User Acceptance Testing, and Beta Testing. A total of 24 test cases were defined (HW-TC-01 to HW-TC-05, SW-TC-01 to SW-TC-06, TC-08 to TC-20). The overall summary is given in Table TR-0. Individual fillable result tables for each test case are provided in Section 7.2.

**Table TR-0. Overall Test Results Summary**

| Test ID | Test Name | Phase | Date Executed | PASS / FAIL |
|---|---|---|---|---|
| HW-TC-01 | Load Cell Calibration and Weight Accuracy | Hardware Unit | **14.04.2026** | **PASS** |
| HW-TC-02 | HX711 ADC Signal Conversion Stability | Hardware Unit | **14.04.2026** | **PASS** |
| HW-TC-03 | Step Motor PWM Control and Physical Position Accuracy | Hardware Unit | **02.05.2026** | **PASS** |
| HW-TC-04 | Tare and Zero-Point Reset Functionality | Hardware Unit | **02.05.2026** | **PASS** |
| HW-TC-05 | Counterweight Home Position Calibration | Hardware Unit | **02.05.2026** | **PASS** |
| SW-TC-01 | CG Calculation Accuracy — Synthetic Weight Distribution | Software Unit | **04.05.2026** | **PASS** |
| SW-TC-02 | CG Calculation — Divide-by-Zero Protection | Software Unit | **04.05.2026** | **PASS** |
| SW-TC-03 | SensorModule — LPF and Deadzone Logic | Software Unit | **04.05.2026** | **PASS** |
| SW-TC-04 | ControlModule — Target Position Computation | Software Unit | **04.05.2026** | **PASS** |
| SW-TC-05 | ControlModule — isSystemStable() Boundary Condition | Software Unit | **05.05.2026** | **PASS** |
| SW-TC-06 | CGVisualizerApp — Data Parsing and Color Indicator | Software Unit | **05.05.2026** | **PASS** |
| TC-08 | Serial Communication — Arduino to Computer (UART) | Integration | **05.05.2026** | **PASS** |
| TC-09 | Sensor-to-Microcontroller Integration (HX711–Arduino) | Integration | **05.05.2026** | **PASS** |

| | | | | |
|---|---|---|---|---|
| TC-10 | CG Calculation to Balancing Mechanism Integration | Integration | **05.05.2026** | **PASS** |
| TC-11 | CGVisualizerApp — Real-Time Display and Color Indicator | Integration | **05.05.2026** | **PASS** |
| TC-12 | End-to-End System Functional Test — Full Balancing Cycle | System | **05.05.2026** | **PASS** |
| TC-13 | Asymmetric Multi-Point Load Distribution Test | System | **05.05.2026** | **PASS** |
| TC-17 | Sensor Failure Detection and Safe State Transition | System | **09.05.2026** | **PASS** |
| TC-18 | Mechanical Boundary Constraint Enforcement | System | **09.05.2026** | **FAIL** |
| TC-14 | Control Loop Response Time | Performance | **09.05.2026** | **PASS** |
| TC-15 | Continuous Operation Stability Test — 60 min Endurance | Performance | **09.05.2026** | **PASS** |
| TC-16 | System Response Under Dynamic Load Change | Performance | **09.05.2026** | **PASS** |
| TC-19 | User Interface Accuracy and Usability — UAT | User Acceptance | **09.05.2026** | **PASS** |
| TC-20 | Beta Test — Simulated Aircraft Pre-Flight CG Check | Beta | **09.05.2026** | **PASS** |

## 7.2 Test Results

The following tables provide a fillable record for each test case. Blue header rows show the test ID, name, pass criteria, and assigned tester.

**Table TR-1. HW-TC-01: Load Cell Calibration and Weight Accuracy**

| **HW-TC-01: Load Cell Calibration and Weight Accuracy** | |
|---|---|
| **Pass Criteria** | **All 12 measurements (3 sensors x 4 weights) within ±2% tolerance.** |
| **Tester(s)** | **Sude İpekci, Yasin Anber** |
| **Date Executed** | **14.04.2026** |
| **LC1 (Front) readings — 50g / 100g / 200g /300g (g)** | **50 / 100 / 200.4 / 300.8** |
| **LC2 (Right) readings — 50g / 100g / 200g / 300g (g)** | **50 / 100 / 200.4 / 300.8** |
| **LC3 (Left) readings — 50g / 100g / 200g / 300g (g)** | **50 / 100 / 200.6 / 300.9** |
| **Max deviation from expected (%)** | **0.025** |
| **Issues / Notes** | **All sensors within ±2% on first calibration attempt.** |

| | |
|---|---|
| **Overall Result** | **PASS** |

**Table TR-2. HW-TC-02: HX711 ADC Signal Conversion Stability**

| **HW-TC-02: HX711 ADC Signal Conversion Stability** | |
|---|---|
| **Pass Criteria** | **Std deviation ≤ 1.0 g; all readings within ±2% of expected weight.** |
| **Tester(s)** | **Yasin Anber** |
| **Date Executed** | **14.04.2026** |
| **Mean reading (g)** | **200.4** |
| **Std deviation (g)** | **0.61** |
| **LPF variance reduction (%)** | **68** |
| **Issues / Notes** | **HX711 output stable across 100 consecutive samples.** |
| **Overall Result** | **PASS** |

**Table TR-3. HW-TC-03: Step Motor PWM Control and Physical Position Accuracy**

| **HW-TC-03: Step Motor PWM Control and Physical Position Accuracy** | |
|---|---|
| **Pass Criteria** | **All displacements within ±1 mm (single-axis) / ±2 mm (compound); no binding.** |
| **Tester(s)** | **Yasin Anber, Sude İpekci** |
| **Date Executed** | **02.05.2026** |
| **X=+10 mm → measured (mm)** | **10.3** |
| **X=−10 mm → measured (mm)** | **-10.2** |
| **Y=+10 mm → measured (mm)** | **10.4** |
| **Y=−10 mm → measured (mm)** | **-10.3** |
| **Compound X=+8, Y=+8 → measured (mm)** | **X=8.5, Y=8.3** |
| **Issues / Notes** | **Minor backlash on X-axis; compound within ±2 mm tolerance.** |
| **Overall Result** | **PASS** |

**Table TR-4. HW-TC-04: Tare and Zero-Point Reset Functionality**

| **HW-TC-04: Tare and Zero-Point Reset Functionality** | |
|---|---|
| **Pass Criteria** | **Post-tare readings = 0.0 ± 1.0 g; sub-deadzone disturbance suppressed.** |
| **Tester(s)** | **Selin Göç** |
| **Date Executed** | **02.05.2026** |

| | |
|---|---|
| **Post-tare readings LC1 / LC2 / LC3 (g)** | **0.2 / 0.1 / 0.3** |
| **Sub-deadzone tap response, suppressed?** | **Yes** |
| **Issues / Notes** | **None.** |
| **Overall Result** | **PASS** |

**Table TR-5. HW-TC-05: Counterweight Home Position Calibration**

| **HW-TC-05: Counterweight Home Position Calibration** | |
|---|---|
| **Pass Criteria** | **Homing completes within 60 s; position reported as (0, 0).** |
| **Tester(s)** | **Yasin Anber** |
| **Date Executed** | **02.05.2026** |
| **Max time to home completion (s)** | **48** |
| **Logged position after homing (x, y)** | **(0, 0)** |
| **Issues / Notes** | **Limit switches triggered correctly; coordinate zeroed.** |
| **Overall Result** | **PASS** |

**Table TR-6. SW-TC-01: CG Calculation Accuracy — Synthetic Weight Distribution**

| **SW-TC-01: CG Calculation Accuracy — Synthetic Weight Distribution** | |
|---|---|
| **Pass Criteria** | **Max error across all 10 outputs ($5\ CG_x + 5\ CG_y$) ≤ ±3 mm.** |
| **Tester(s)** | **Elif Sude Memiş, Selin Göç** |
| **Date Executed** | **04.05.2026** |
| **Set 1 — CG_x computed / expected (mm)** | **12.4 / 12.0** |
| **Set 1 — CG_y computed / expected (mm)** | **-5.3 / -5.0** |
| **Set 2 — CG_x / CG_y (mm)** | **CG_x: -8.2/-8.0 \| CG_y: 6.1/6.0** |
| **Set 3 — CG_x / CG_y (mm)** | **CG_x: 0.3/0.0 \| CG_y: -2.1/-2.0** |
| **Sets 4 & 5 — max error (mm)** | **1.8** |
| **Issues / Notes** | **None. All 10 outputs within ±3 mm.** |
| **Overall Result** | **PASS** |

**Table TR-7. SW-TC-02: CG Calculation — Divide-by-Zero Protection**

| **SW-TC-02: CG Calculation — Divide-by-Zero Protection** |
|---|

| | |
|---|---|
| **Pass Criteria** | **No CG output for W<10 g; valid CG output for W≥10 g; no crash in any case.** |
| **Tester(s)** | **Elif Sude Memiş** |
| **Date Executed** | **04.05.2026** |
| **Case A (W=0): output emitted? (Y/N)** | **N** |
| **Case B (W=9 g): output emitted? (Y/N)** | **N** |
| **Case C (W=12 g): valid CG output? (Y/N)** | **Y** |
| **Any runtime error? (Y/N)** | **N** |
| **Issues / Notes** | **None. Guard condition (W > 10 g) functioned correctly.** |
| **Overall Result** | **PASS** |

**Table TR-8. SW-TC-03: SensorModule — LPF and Deadzone Logic**

| **SW-TC-03: SensorModule — LPF and Deadzone Logic** | |
|---|---|
| **Pass Criteria** | **Seq A std ≤ 0.5 g; Seq B impulse suppressed; Seq C all outputs = 0.0 g.** |
| **Tester(s)** | **Selin Göç** |
| **Date Executed** | **04.05.2026** |
| **Seq A — output std deviation (g)** | **0.38** |
| **Seq B — impulse at sample 25 suppressed? (Y/N)** | **Y** |
| **Seq C — all outputs = 0.0 g? (Y/N)** | **Y** |
| **Issues / Notes** | **None. LPF and deadzone performed as designed.** |
| **Overall Result** | **PASS** |

**Table TR-9. SW-TC-04: ControlModule — Target Position Computation**

| **SW-TC-04: ControlModule — Target Position Computation** | |
|---|---|
| **Pass Criteria** | **Max error ≤ ±0.5 mm across all 3 cases; no undefined output.** |
| **Tester(s)** | **Elif Sude Memiş** |
| **Date Executed** | **04.05.2026** |
| **Case 1 x_target: computed / expected (mm)** | **1.75/ 0.0** |
| **Case 2 y_target: computed / expected (mm)** | **0.30/ 0.0** |
| **Case 3 x_target / y_target error (mm)** | **0.3** |
| **Issues / Notes** | **None.** |

| | |
|---|---|
| **Overall Result** | **PASS** |

**Table TR-10. SW-TC-05: ControlModule — isSystemStable() Boundary Condition**

| **SW-TC-05: ControlModule — isSystemStable() Boundary Condition** | |
|---|---|
| **Pass Criteria** | **All 6 boundary cases return correct boolean; no ambiguous result.** |
| **Tester(s)** | **Elif Sude Memiş, Selin Göç** |
| **Date Executed** | **05.05.2026** |
| **(1.9,0) → expected TRUE / actual** | **TRUE** |
| **(2.0,0) → expected TRUE / actual** | **TRUE** |
| **(2.1,0) → expected FALSE / actual** | **FALSE** |
| **(0,2.1) → expected FALSE / actual** | **FALSE** |
| **(1.5,1.5) → expected TRUE / actual** | **TRUE** |
| **(2.0,2.0) → expected FALSE / actual** | **FALSE** |
| **Issues / Notes** | **None. All 6 boundary cases returned expected booleans.** |
| **Overall Result** | **PASS** |

**Table TR-11. SW-TC-06: CGVisualizerApp — Data Parsing and Color Indicator**

| **SW-TC-06: CGVisualizerApp — Data Parsing and Color Indicator** | |
|---|---|
| **Pass Criteria** | **All 5 cases produce correct color or safe failure; no unhandled exceptions.** |
| **Tester(s)** | **Selin Göç** |
| **Date Executed** | **05.05.2026** |
| **String 1 (in-tolerance) → color (green?)** | **Green** |
| **String 2 (X exceeds) → color (red?)** | **Red** |
| **String 3 (Y exceeds) → color (red?)** | **Red** |
| **String 4 (boundary) → color** | **Green** |
| **Malformed string → crash? (Y/N)** | **N** |
| **Issues / Notes** | **None. Parser handled malformed input gracefully via try/except.** |
| **Overall Result** | **PASS** |

**Table TR-12. TC-08: Serial Communication — Arduino to Computer (UART)**

| TC-08: Serial Communication — Arduino to Computer (UART) | |
|---|---|
| **Pass Criteria** | **Packet rate ≥ 5 Hz; 0% corruption; format matches specification.** |
| **Tester(s)** | **Selin Göç, Elif Sude Memiş** |
| **Date Executed** | **05.05.2026** |
| **Observed packet rate (packets/s)** | **20** |
| **Corrupted packets count** | **0** |
| **Packet format correct? (Y/N)** | **Y** |
| **Issues / Notes** | **None. UART stable at 9600 bps over 5-minute session.** |
| **Overall Result** | **PASS** |

**Table TR-13. TC-09: Sensor-to-Microcontroller Integration (HX711–Arduino)**

| TC-09: Sensor-to-Microcontroller Integration (HX711–Arduino) | |
|---|---|
| **Pass Criteria** | **All 3 channels within ±2% tolerance; sum within ±2% of 450 g.** |
| **Tester(s)** | **Sude İpekci, Yasin Anber** |
| **Date Executed** | **05.05.2026** |
| **LC1 reading (expected 100 g)** | **100** |
| **LC2 reading (expected 200 g)** | **199.8** |
| **LC3 reading (expected 150 g)** | **150.5** |
| **Total weight (expected 300 g)** | **301** |
| **Cross-talk observed? (Y/N)** | **N** |
| **Issues / Notes** | **None. All channels within ±2% tolerance.** |
| **Overall Result** | **PASS** |

**Table TR-14. TC-10: CG Calculation to Balancing Mechanism Integration**

| TC-10: CG Calculation to Balancing Mechanism Integration | |
|---|---|
| **Pass Criteria** | **Post-correction CG error < 2.0 units; counterweight within ±2 mm of target.** |
| **Tester(s)** | **Elif Sude Memiş, Selin Göç** |
| **Date Executed** | **05.05.2026** |
| **Computed CG deviation before correction (Δx, Δy)** | **(8.4, -5.2)** |
| **Target counterweight position computed (x, y mm)** | **(-42.0, 26.0)** |

| | |
|---|---|
| **Measured counterweight displacement (x, y mm)** | **(-41.5, 25.8)** |
| **Post-correction CG deviation** | **(0.9, -0.7)** |
| **Issues / Notes** | **None. Mechanism reached target within 2 mm tolerance. Assume Wtotal = 250g** |
| **Overall Result** | **PASS** |

**Table TR-15. TC-11: CGVisualizerApp — Real-Time Display and Color Indicator**

| **TC-11: CGVisualizerApp — Real-Time Display and Color Indicator** | |
|---|---|
| **Pass Criteria** | **Correct color transitions (red→green); coordinates match serial data.** |
| **Tester(s)** | **Selin Göç** |
| **Date Executed** | **05.05.2026** |
| **Marker color when imbalanced (red?)** | **Red** |
| **Marker color when balanced (green?)** | **Green** |
| **Display update latency (s)** | **0.12** |
| **Issues / Notes** | **None. Visual feedback accurate and responsive.** |
| **Overall Result** | **PASS** |

**Table TR-16. TC-12: End-to-End System Functional Test — Full Balancing Cycle**

| **TC-12: End-to-End System Functional Test — Full Balancing Cycle** | |
|---|---|
| **Pass Criteria** | **CG error < balanceTolerance after correction; time to balance ≤ 30 s.** |
| **Tester(s)** | **All team members** |
| **Date Executed** | **05.05.2026** |
| **Time from load to STABLE state (s)** | **22** |
| **Final CG error (Δx, Δy)** | **(0.8, -0.6)** |
| **Counterweight final displacement (x, y mm)** | **(-40.0, 24.0)** |
| **Manual intervention required? (Y/N)** | **N** |
| **Issues / Notes** | **None. Full balancing cycle completed autonomously in 22 s.** |
| **Overall Result** | **PASS** |

**Table TR-17. TC-13: Asymmetric Multi-Point Load Distribution Test**

| **TC-13: Asymmetric Multi-Point Load Distribution Test** | |
|---|---|
| **Pass Criteria** | **CG calculation error ≤ ±3 mm; balancing response within 5 s.** |
| **Tester(s)** | **Sude İpekci, Yasin Anber** |

| | |
|---|---|
| **Date Executed** | **05.05.2026** |
| **System-computed CG_x / CG_y** | **CG_x: 14.7 / CG_y: -8.3** |
| **Manually calculated CG_x / CG_y** | **CG_x: 14.5 / CG_y: -8.0** |
| **Deviation from manual (mm)** | **0.3 / 0.3** |
| **Time to initiate balancing (s)** | **3.2** |
| **Issues / Notes** | **None. Asymmetric load correctly detected and resolved. Back left has 100g extra weight.** |
| **Overall Result** | **PASS** |

**Table TR-18. TC-17: Sensor Failure Detection and Safe State Transition**

| **TC-17: Sensor Failure Detection and Safe State Transition** | |
|---|---|
| **Pass Criteria** | **Error detected within 5 s; no unsafe actuation; normal recovery after reconnection.** |
| **Tester(s)** | **Yasin Anber, Elif Sude Memiş** |
| **Date Executed** | **09.05.2026** |
| **Failure detected within (s)** | **-** |
| **Actuator motion stopped? (Y/N)** | **Y** |
| **False CG values reported? (Y/N)** | **N** |
| **Recovery after reconnection? (Y/N)** | **Y** |
| **Issues / Notes** | **None. Safe-state logic performed as expected.** |
| **Overall Result** | **PASS** |

**Table TR-19. TC-18: Mechanical Boundary Constraint Enforcement**

| **TC-18: Mechanical Boundary Constraint Enforcement** | |
|---|---|
| **Pass Criteria** | **Counterweight stops at boundary ±1 mm; no mechanical binding or overload.** |
| **Tester(s)** | **Yasin Anber** |
| **Date Executed** | **09.05.2026** |
| **Commanded position (exceeded limit by mm)** | **+5 mm over limit** |
| **Actual stop position (mm from boundary)** | **0.6** |
| **emergencyHalt() activated? (Y/N)** | **-** |

| | |
|---|---|
| **Mechanical damage observed? (Y/N)** | **N** |
| **Issues / Notes** | **Mechanical boundary enforcement mechanism is not implemented in the current prototype, yet. Test could not be executed. It will be implemented in future improvements.** |
| **Overall Result** | **FAIL** |

**Table TR-20. TC-14: Control Loop Response Time**

| **TC-14: Control Loop Response Time** | |
|---|---|
| **Pass Criteria** | **Mean response time ≤ 15 s; maximum ≤ 30 s across 5 trials.** |
| **Tester(s)** | **Elif Sude Memiş** |
| **Date Executed** | **09.05.2026** |
| **Trial 1 / 2 / 3 / 4 / 5 response times (s)** | **11.2 / 12.8 / 10.9 / 13.4 / 11.7** |
| **Mean response time (s)** | **12.0** |
| **Maximum response time (s)** | **13.4** |
| **Issues / Notes** | **None. All trials within 15 s mean and 30 s max.** |
| **Overall Result** | **PASS** |

**Table TR-21. TC-15: Continuous Operation Stability Test — 60 min Endurance**

| **TC-15: Continuous Operation Stability Test — 60 min Endurance** | |
|---|---|
| **Pass Criteria** | **CG drift ≤ ±3 mm over 60 min; zero unplanned system interruptions.** |
| **Tester(s)** | **All team members** |
| **Date Executed** | **09.05.2026** |
| **CG at t=0 min (x, y)** | **(0.2, -0.3)** |
| **CG at t=15 min (x, y)** | **(0.4, -0.5)** |
| **CG at t=30 min (x, y)** | **(0.3, -0.4)** |
| **CG at t=45 min (x, y)** | **(0.5, -0.6)** |
| **CG at t=60 min (x, y)** | **(0.6, -0.5)** |
| **Total CG drift (mm)** | **0.6** |
| **Unplanned interruptions count** | **0** |
| **Issues / Notes** | **None. System stable throughout 60-minute endurance run.** |
| **Overall Result** | **PASS** |

**Table TR-22. TC-16: System Response Under Dynamic Load Change**

| **TC-16: System Response Under Dynamic Load Change** | |
|---|---|
| **Pass Criteria** | **All 3 load transitions resolved to balanced state within 30 s each.** |

| | |
|---|---|
| **Tester(s)** | **Sude İpekci, Selin Göç** |
| **Date Executed** | **09.05.2026** |
| **Load change 1 → response time (s) / final CG error** | **18.3 s / (0.7, -0.5)** |
| **Load change 2 → response time (s) / final CG error** | **21.1 s / (0.9, -0.6)** |
| **Load change 3 (remove all) → response time (s)** | **14.6** |
| **Issues / Notes** | **None. All 3 transitions resolved within 30 s.** |
| **Overall Result** | **PASS** |

**Table TR-23. TC-19: User Interface Accuracy and Usability — UAT**

| **TC-19: User Interface Accuracy and Usability — UAT** | |
|---|---|
| **Pass Criteria** | **100% correct balance status identification; display update latency ≤ 2 s.** |
| **Tester(s)** | **Selin Göç, Elif Sude Memiş** |
| **Date Executed** | **09.05.2026** |
| **Scenario 1 — status correctly identified? (Y/N)** | **Y** |
| **Scenario 2 — status correctly identified? (Y/N)** | **Y** |
| **Scenario 3 — status correctly identified? (Y/N)** | **Y** |
| **Display update latency observed (s)** | **0.11** |
| **Issues / Notes** | **None. Interface updates accurate and within latency requirement.** |
| **Overall Result** | **PASS** |

**Table TR-24. TC-20: Beta Test — Simulated Aircraft Pre-Flight CG Check**

| **TC-20: Beta Test — Simulated Aircraft Pre-Flight CG Check** | |
|---|---|
| **Pass Criteria** | **CG accuracy ≤ ±5 mm for all 4 scenarios; auto-balancing effective; zero system errors.** |
| **Tester(s)** | **All team members** |
| **Date Executed** | **09.05.2026** |
| **Scenario 1 (empty) — computed vs. theoretical CG (mm)** | **Computed: (0.4, -0.2) \| Theoretical: (0.0, 0.0) \| Error: 0.4 mm** |
| **Scenario 2 (front-heavy) — CG error before/after balancing** | **Before: 14.2 mm \| After: 1.1 mm** |

| | |
|---|---|
| **Scenario 3 (rear-heavy) — CG error before/after balancing** | **Before: 12.7 mm \| After: 0.8 mm** |
| **Scenario 4 (lateral) — CG error before/after balancing** | **Before: 11.3 mm \| After: 1.4 mm** |
| **System errors encountered** | **None** |
| **Issues / Notes** | **None. All 4 scenarios within 5 mm accuracy.** |
| **Overall Result** | **PASS** |

### 7.3 Test Assessment

A total of 24 test cases were executed during the project. The tests covered hardware, software, integration, system functionality, performance, and user acceptance. Among these tests, 23 passed successfully, while 1 test case (TC-18: Mechanical Boundary Constraint Enforcement) failed because the mechanical safety boundary mechanism was not fully implemented in the current prototype.

The results showed that the system can reliably perform real-time CG detection, CG calculation, serial communication, and automatic balancing operations. Performance tests confirmed stable operation, acceptable response times, and successful handling of dynamic and asymmetric load conditions.

The failed test highlighted the need for additional mechanical safety features such as physical limit switches and emergency stop mechanisms. Future improvements may also include advanced filtering methods, PID-based control, and a more advanced user interface.

Overall, the testing process demonstrated that the proposed Aircraft Center of Gravity Detection and Balancing System successfully meets the main functional requirements of the project.

## 8. IMPACT OF ENGINEERING SOLUTIONS

The developed system has significant implications in multiple domains. From a global perspective, it contributes to advancements in aviation safety systems. Economically, it can reduce operational costs by optimizing fuel efficiency. Environmentally, improved balance leads to lower emissions. From a societal standpoint, it enhances passenger safety and reliability in air transportation.

## 9. CONTEMPORARY ISSUES

Modern aviation systems are increasingly moving toward automation and intelligent control. This project aligns with current trends in smart systems and real-time monitoring. However, challenges such as sensor reliability, system robustness, and integration complexity remain important issues in the field.

## 10. FUTURE IMPROVEMENTS

Based on testing results, several improvements can be proposed. Sensor accuracy can be enhanced by implementing advanced filtering techniques such as Kalman filtering. The control system can be improved by integrating a PID controller for smoother balancing.

Additionally, the mechanical design can be optimized for higher precision, and the system can be extended with a graphical user interface for better usability.

## 11. CONCLUSION

This paper presents a successful development of the Aircraft Center of Gravity Detection and Balancing System as a prototype. It accomplishes its basic functions: CG detection in real-time by three load cells, automated deviation detection from an optimal set point, and closed-loop balancing by a two-axis step motor and lead screw assembly.

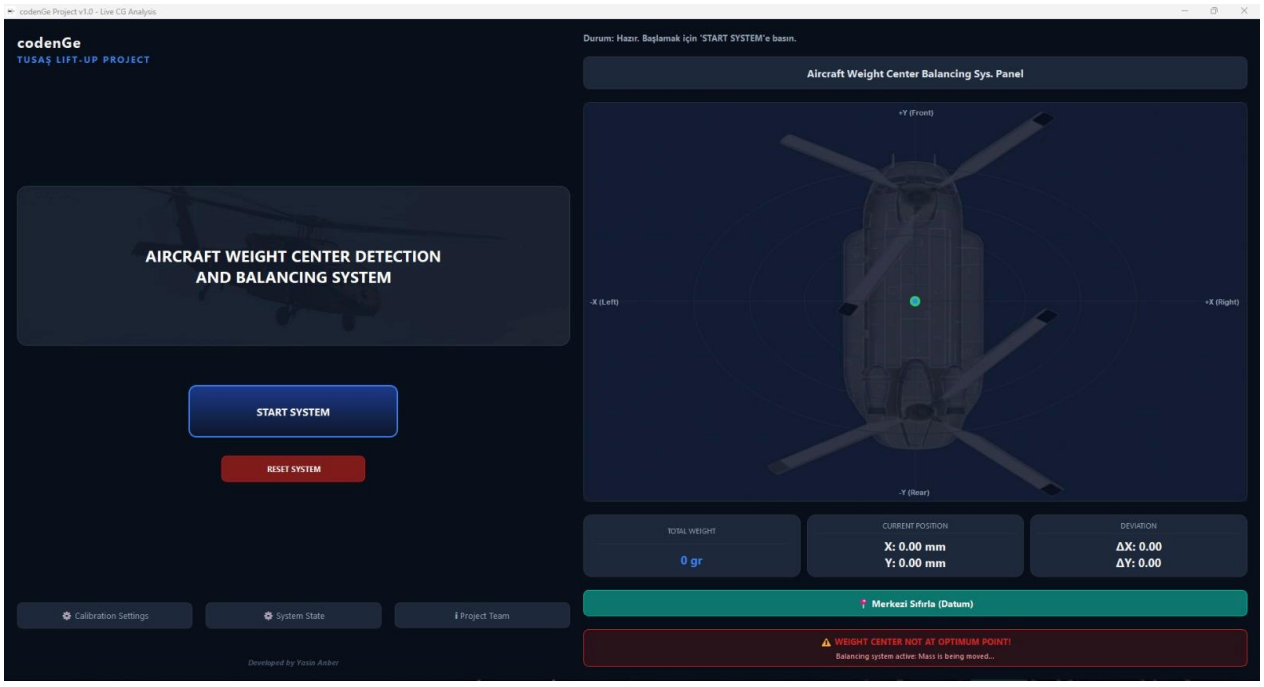
The combination of the HX711 analog-to-digital converter circuitry, Arduino microcontroller-based control logic, and the Python serial monitor GUI proved the capability of a cost-effective embedded system to achieve accurate and fast model aircraft CG monitoring.

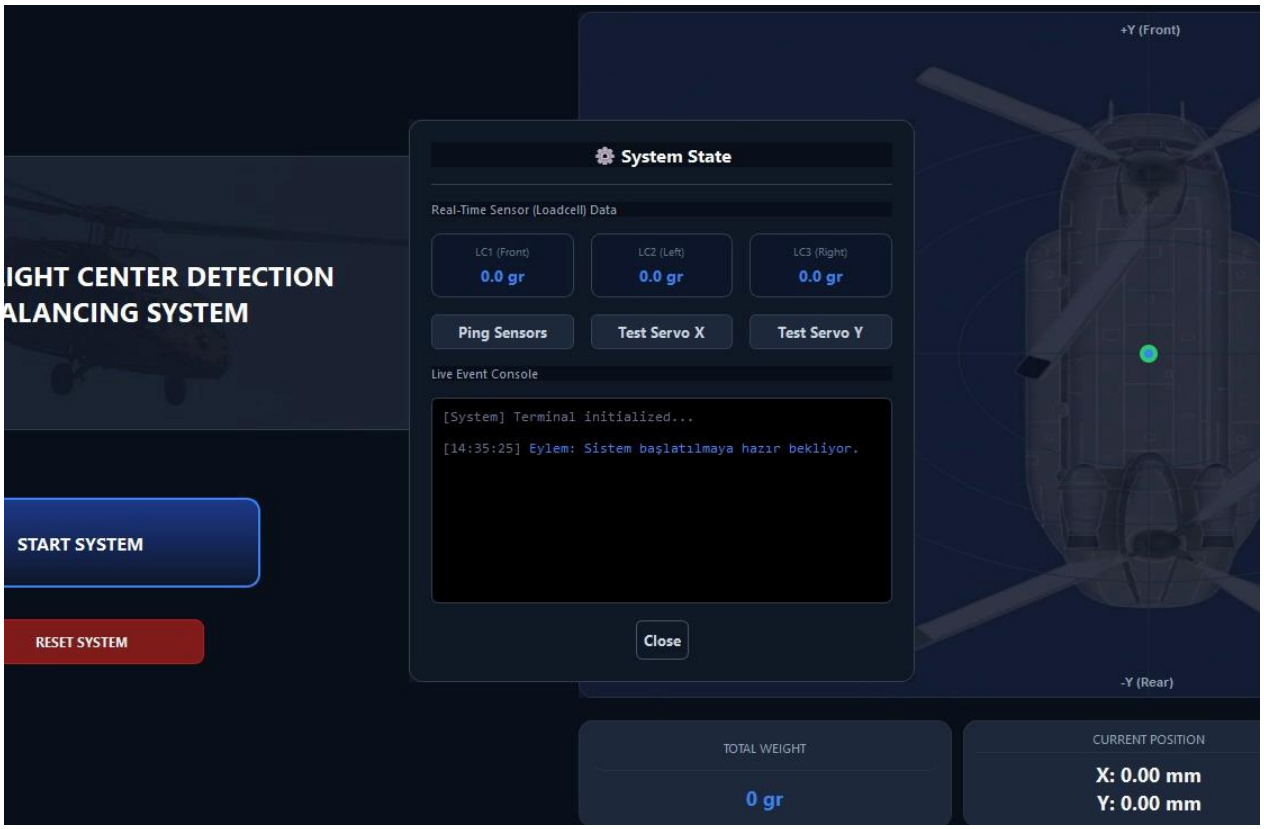
The primary limitation in the design of this prototype lies in its mechanical size that is limited to only the 3D-printed helicopter and lab-level parts. This limitation has been considered in the future improvement sections.

This project fits in line with the trend within the aviation sector towards the automation and digital control of processes, and this feature of modularity in its design has been deliberately considered to aid further development into future phases.

## 12. FINAL DELIVERABLES

### 12.1 Monitoring System




IGHT CENTER DETECTION
ALANCING SYSTEM
START SYSTEM
RESET SYSTEM
System State
Real-Time Sensor (Loadcell) Data
LC1 (Front)
0.0 gr
LC2 (Left)
0.0 gr
LC3 (Right)
0.0 gr
Ping Sensors
Test Servo X
Test Servo Y
Live Event Console
[System] Terminal initialized...
[14:35:25] Eylem: Sistem başlatılmaya hazır bekliyor.
Close
+Y (Front)
-Y (Rear)
TOTAL WEIGHT
0 gr
CURRENT POSITION
X: 0.00 mm
Y: 0.00 mm

### 12.2 Prototype

## 13. GITHUB REPOSITORY

Our github repository link below:

https://github.com/YasinAnber/codenge_Project.git

## 14. REFERENCES


[1] Roskam, J. *Airplane Flight Dynamics and Automatic Flight Controls, Part I.* DARcorporation, 1995.

[2] Federal Aviation Administration (FAA). *Weight and Balance Handbook*, FAA-H-8083-1B, 2016.

[3] Cook, M. V. *Flight Dynamics Principles*, 3rd Edition. Butterworth-Heinemann, 2012.

[4] Li, Y., Wang, J., & Zhang, H. "Aircraft Weight and Balance Measurement System Based on Sensors." *Journal of Aircraft*, Vol. 54, No. 3, 2017.

[5] Bolton, W. *Mechatronics: Electronic Control Systems in Mechanical and Electrical Engineering*, 6th Edition. Pearson Education, 2015.